

Technical Analysis Document

DocRestoreAI: Generative Document Restoration and PDF Enhancement

1. Problem Statement

Many documents are scanned or photographed in poor conditions. This often creates visual problems such as noise, blur, low contrast, shadows, skewed pages, dirty backgrounds, and reduced text readability.

These problems are common in schools, offices, archives, administrative documents, and old scanned PDFs. Poor document quality can make reading difficult and can also reduce the performance of OCR systems.

The goal of this project is to build a Computer Vision application that restores low-quality document images and scanned PDFs. The system receives a damaged document as input and generates a cleaner, more readable version as output.

Input: damaged / noisy / low-quality document image or PDF
Output: restored / enhanced document image or PDF

The project is relevant because document restoration can improve digital archiving, readability, and accessibility.

2. Methodology

The project follows a complete Computer Vision pipeline composed of four main stages:

1. Data acquisition and preprocessing
2. Feature engineering and representation
3. Core generative model
4. Post-processing and evaluation

The system combines classical Computer Vision techniques with a deep learning image-to-image translation model.

2.1 Data Acquisition and Preprocessing

The dataset is created using clean document images. Since paired damaged-clean datasets are difficult to find, the project generates synthetic damaged images from clean documents.

Each clean document image is used as the target image. A damaged version is generated automatically and used as the input image.

The degradation pipeline includes:

- Gaussian noise
- blur
- rotation
- low contrast
- artificial shadows
- scanning-like visual damage

This creates paired training data:

Damaged document image -> Clean document image

Preprocessing operations include:

- loading images using OpenCV
- converting images to RGB format
- resizing images to a fixed size
- normalizing pixel values to the range $[-1, 1]$
- converting images to PyTorch tensors
- converting PDF pages into images when needed

Normalization is important because the Generator uses a tanh output activation, which produces values between -1 and 1.

2.2 Feature Engineering and Representation

The project includes both classical Computer Vision features and learned deep learning features.

Classical Features

Classical feature extraction is used to analyze document quality before and after restoration.

The extracted features include:

Feature	Purpose
HOG	Represents document edges and text structure
Canny edges	Detects text and page boundaries
Contour count	Estimates connected text/document regions
Sharpness	Measures blur using Laplacian variance
Brightness	Measures average image intensity
Contrast	Measures text-background separation
Dark pixel ratio	Estimates visible text density

These features are useful for explaining whether the restoration improves sharpness, contrast, and text readability.

Learned Features

The deep learning model learns features automatically through convolutional layers.

The Generator learns visual representations of:

- document layout
- text edges
- background noise
- shadows
- blur patterns
- clean document structure

This allows the model to transform a damaged document image into a restored version.

2.3 Core Model Architecture

The main model is a Pix2Pix-style Conditional GAN.

Pix2Pix is suitable for this project because document restoration is a paired image-to-image translation problem.

```
Input domain: damaged documents
Output domain: clean documents
```

The model contains two neural networks:

1. Generator
2. Discriminator

Generator: U-Net

The Generator uses a U-Net architecture.

The U-Net contains:

- encoder blocks
- bottleneck layer
- decoder blocks
- skip connections

The encoder compresses the damaged document image and learns high-level features.

The decoder reconstructs the restored image.

Skip connections are important because they preserve low-level details such as text edges, page layout, and document structure.

The Generator output is a restored document image.

Discriminator: PatchGAN

The Discriminator uses a PatchGAN architecture.

Instead of classifying the whole image as real or fake, PatchGAN classifies small image patches.

This is useful for document restoration because local details are important, such as:

- sharp text edges
- clean background
- reduced noise
- natural document texture

The Discriminator receives both the damaged input image and either the clean target or generated output.

2.4 Training Strategy

The model is trained using two losses:

Adversarial Loss

The adversarial loss encourages the Generator to create realistic restored documents that can fool the Discriminator.

L1 Reconstruction Loss

The L1 loss compares the generated image with the clean target image.

This helps the Generator produce outputs that are visually close to the clean document.

The final Generator loss is:

$$\text{Generator Loss} = \text{GAN Loss} + \lambda_{L1} * \text{L1 Loss}$$

In this project, λ_{L1} is used to give strong importance to reconstruction quality.

2.5 Post-processing

After the Generator produces the restored image, optional classical Computer Vision post-processing is applied.

Post-processing includes:

- denormalization
- pixel clipping
- denoising
- contrast enhancement
- deskew correction
- sharpening
- PDF reconstruction

For PDF input, the system follows this workflow:

```
PDF -> page images -> restoration model -> restored images -> restored PDF
```

This makes the system usable not only for single images but also for scanned PDF documents.

3. Experimental Results

The model is evaluated using metrics suitable for image restoration and generative image-to-image translation.

The main metrics are:

Metric	Meaning	Goal
PSNR	Pixel-level reconstruction quality	Higher is better
SSIM	Structural similarity between generated and clean image	Higher is better
FID	Similarity between generated and real clean image distribution	Lower is better
OCR improvement	Improvement in text readability	Positive value is better
Qualitative analysis	Visual comparison of results	Better readability

3.1 Quantitative Results

After running the evaluation script, the results are saved in:

- outputs/evaluation/metrics/evaluation_summary.txt
- outputs/evaluation/metrics/evaluation_metrics.csv

The following table shows the experimental results obtained from the evaluation pipeline:

Metric	Result
Average PSNR	9.4048
Average SSIM	0.0793
FID Score	Not computed
Average OCR Improvement	Not available

The evaluation also generates visual plots and qualitative comparison images.

Generated files:

- outputs/evaluation/plots/psnr_plot.png
- outputs/evaluation/plots/ssim_plot.png
- outputs/evaluation/qualitative/

These files are used to support the experimental results section.

3.2 Qualitative Results

The qualitative evaluation compares three images:

Damaged Input | Generated Restoration | Clean Target

The comparison images are saved in:

outputs/evaluation/qualitative/

The expected result is that the generated image has:

- cleaner background
- sharper text
- fewer shadows
- reduced noise
- improved contrast
- better readability

4. Failure Analysis

The model may fail in some difficult cases.

Possible failure cases include:

Extremely blurry documents

If the original text is too blurred, the model may not recover fine details correctly.

Very strong shadows

Large shadows may be difficult to remove completely, especially if they cover important text areas.

Low-resolution scans

If the input image has very low resolution, the model has limited visual information to reconstruct.

Handwritten documents

Handwriting varies strongly between people. If the model is trained mostly on printed documents, it may perform worse on handwritten notes.

Complex backgrounds

Documents photographed on textured or cluttered backgrounds may confuse the model.

Missing content

The model can improve readability, but it cannot reliably reconstruct text that is completely missing or covered.

Since this is a generative model, it may sometimes produce visually plausible results that are not perfectly faithful to the original document.

5. Ethical Considerations

This project is designed for document restoration and readability enhancement only.

It should not be used to:

- falsify official documents
- change dates
- alter names
- modify signatures
- manipulate legal or administrative content
- create misleading documents

The restored document should be considered an enhanced visual version, not proof of original authenticity.

There are also privacy concerns. Uploaded documents may contain personal, legal, financial, or sensitive information. For this reason, the system should avoid storing user documents unnecessarily.

Bias can also appear if the model is trained only on a narrow type of document. For example, if the dataset contains mostly printed English documents, the model may perform worse on handwritten documents or documents in other writing systems.

To reduce ethical risks, the application includes a clear statement explaining that the tool is intended only for readability improvement and not for document manipulation.

6. Conclusion

DocRestoreAI demonstrates a complete Computer Vision pipeline for generative document restoration and PDF enhancement.

The project includes:

- data acquisition and preprocessing
- synthetic degradation generation
- classical feature extraction

- Pix2Pix GAN model implementation
- post-processing
- PDF input/output support
- performance evaluation
- Streamlit web application

The system shows how classical Computer Vision and deep learning can be combined to solve a real-world problem.

Future improvements could include:

- training on a larger real-world scanned document dataset
- adding OCR-based text preservation loss
- improving PDF page layout reconstruction
- deploying the system as a web API
- supporting multi-language document restoration

7. References

- Pix2Pix: Image-to-Image Translation with Conditional Adversarial Networks
- U-Net: Convolutional Networks for Biomedical Image Segmentation
- OpenCV Documentation
- PyTorch Documentation
- scikit-image Documentation
- Streamlit Documentation